DWE LINK

Le lien entre patients, donateurs et pharmacies

Conception d'une plateforme de gestion des médicaments, des pharmacies et du don solidaire en architecture microservices.

PARCOURS UTILISATEUR SIMPLIFIÉ

01 RECHERCHER
Trouver un médicament

02 RÉSERVER
Mettre de côté les boîtes

03 RETIRER
Présenter le code au comptoir

04 DONNER
Offrir ses médicaments inutilisés

SPRING BOOT • ANGULAR • REST • ÉVÉNEMENTS RABBITMQ • KEYCLOAK

04 ACTEURS DE LA PLATEFORME

07 MICROSERVICES DÉTAILLÉS

06 MEMBRES DE L'ÉQUIPE

ÉQUIPE ET RESPONSABLES

01
Hichem Jemail
Utilisateurs et inscription

02
Mohamed Yassine Ben Aïssa
Gestion des pharmacies

03
Wejdene Bouali
Catalogue des médicaments

04
Rayen Sassi
Stock et alertes

05
Roua Selmi
Gestion des réservations

06
Mohamed Yassine Azzouz
Dons et solidarité

07
Microservice commun
Notifications

SOMMAIRE

Le rapport en sept sections

Chaque section répond à un point attendu du dossier de conception.

01 **Description du sujet** — Contexte, problématique, objectifs et périmètre. PAGE 03

02 **Acteurs et fonctionnalités** — Quatre acteurs, fonctionnalités par module, diagramme des cas d'utilisation. PAGE 04

03 **Architecture globale** — Interface, passerelle API, Keycloak, Eureka, Config Server, microservices, bases et RabbitMQ. PAGE 06

04 **Découpage en microservices** — Une fiche par service et son diagramme de classes. PAGE 06

05 **Données gérées** — Une base de données par service et le choix de chaque technologie. PAGE 21

06 **Interactions entre les services** — Carte des interactions, appels REST et événements RabbitMQ. PAGE 22

07 **Organisation du projet** — Technologies, planning, dépôt Git et risques. PAGE 24

SECTION 01

01 Comprendre DweLink

Le contexte du projet, le problème à résoudre et les objectifs de la plateforme.

CONTEXTE

Les patients, les pharmacies et les personnes qui possèdent des médicaments inutilisés n'ont pas aujourd'hui d'outil commun pour échanger des informations. DweLink est une plateforme web qui les relie : le patient trouve et réserve un médicament, le pharmacien gère sa pharmacie et son stock, et le particulier peut donner gratuitement les médicaments dont il n'a plus besoin.

PROBLÉMATIQUE

Trouver un médicament reste souvent difficile. Le patient ne sait pas quelle pharmacie le possède : il doit appeler plusieurs officines ou se déplacer sans certitude. De leur côté, beaucoup de pharmacies suivent encore leur stock à la main ou avec des outils séparés. Il est alors difficile de repérer à temps une rupture ou un lot qui va bientôt se périmer. Enfin, des médicaments en bon état restent inutilisés chez les particuliers, alors que d'autres personnes n'ont pas les moyens de les acheter.

Information
Le patient ne sait pas où trouver son médicament.

Gestion
Le stock et les péremptions sont mal suivis.

Gaspillage
Des médicaments utilisables ne servent à personne.

> **Comment proposer une plateforme unique qui facilite l'accès aux médicaments, aide les pharmacies à mieux gérer leur stock et organise le don de médicaments de façon sûre ?**

Pour répondre à cette question, DweLink propose une solution centralisée, découpée en microservices. Chaque besoin (utilisateurs, pharmacies, catalogue, stock, réservations, dons, notifications) est confié à un service indépendant. L'application est ainsi plus simple à développer en équipe, à faire évoluer et à maintenir.

OBJECTIFS

- Montrer au patient les pharmacies qui ont son médicament et lui permettre de le réserver.
- Aider le pharmacien à suivre son stock, ses lots et leurs dates de péremption.
- Organiser le don de médicaments, contrôlé par un pharmacien et sans revente.
- Contrôler l'accès à la plateforme : pharmaciens et pharmacies validés par l'administrateur.

PÉRIMÈTRE : SEPT MICROSERVICES

01
Utilisateurs et inscription

`user-service`

Profils, inscription et vérification des pharmaciens, statut des comptes et préférences.

02
Gestion des pharmacies

`pharmacy-service`

Pharmacies, localisation, horaires et gardes, validation administrative et membres.

03
Catalogue des médicaments

`catalog-service`

Référentiel commun des médicaments, géré par l'administrateur et consulté par tous.

04
Stock et alertes

`inventory-service`

Stock de chaque pharmacie, lots, dates de péremption et alertes.

05
Gestion des réservations

`reservation-service`

Réservation par le patient, traitement par la pharmacie, code de retrait, annulation, expiration et historique.

06
Dons et solidarité

`donation-service`

Don gratuit de médicaments par un particulier, contrôle par la pharmacie et remise solidaire.

07
Notifications

`notification-service`

Notifications dans l'application et par email, à partir des événements de tous les services.

Le paiement en ligne, la livraison, les commandes aux fournisseurs et l'intelligence artificielle ne font pas partie du projet.

SECTION 02

02 Acteurs et fonctionnalités

Chaque acteur dispose de fonctionnalités propres selon son rôle : patient, donateur, pharmacien ou administrateur.

01
PATIENT

Recherche et réservation

- Rechercher des médicaments
- Trouver une pharmacie proche
- Réserver un médicament
- Suivre et annuler ses réservations

02
DONATEUR

Don solidaire

- Proposer un don de médicament
- Choisir une pharmacie
- Déposer le médicament
- Suivre et annuler ses dons

03
PHARMACIEN

Gestion de la pharmacie

- Gérer sa pharmacie et ses membres
- Gérer le stock, les lots et les alertes
- Traiter les réservations
- Contrôler et remettre les dons

04
ADMINISTRATEUR

Contrôle de la plateforme

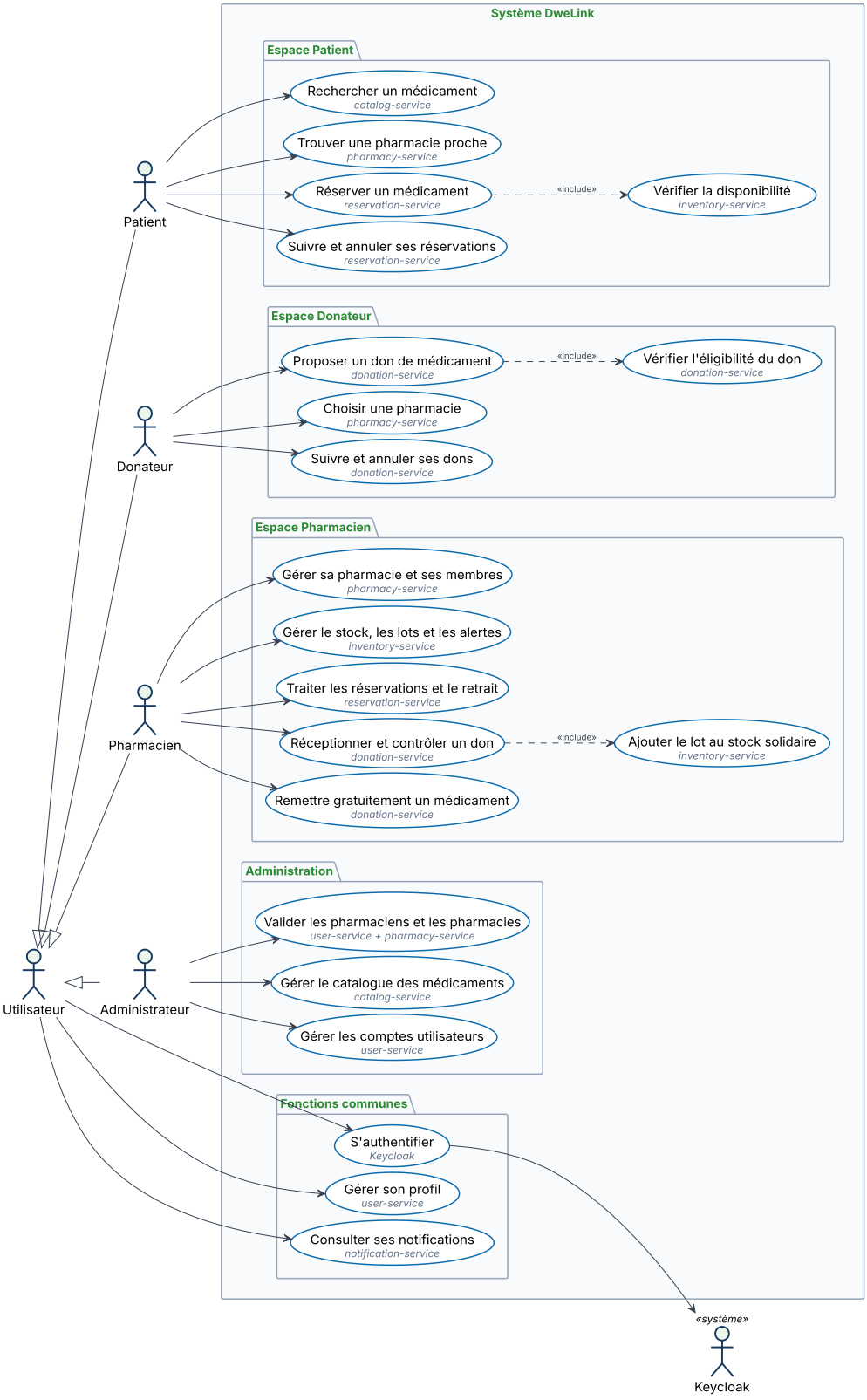
- Valider les pharmaciens
- Valider les pharmacies
- Gérer le catalogue
- Gérer les comptes

SECTION 02

Diagramme des cas d'utilisation

Le microservice concerné est indiqué sous chaque cas d'utilisation.

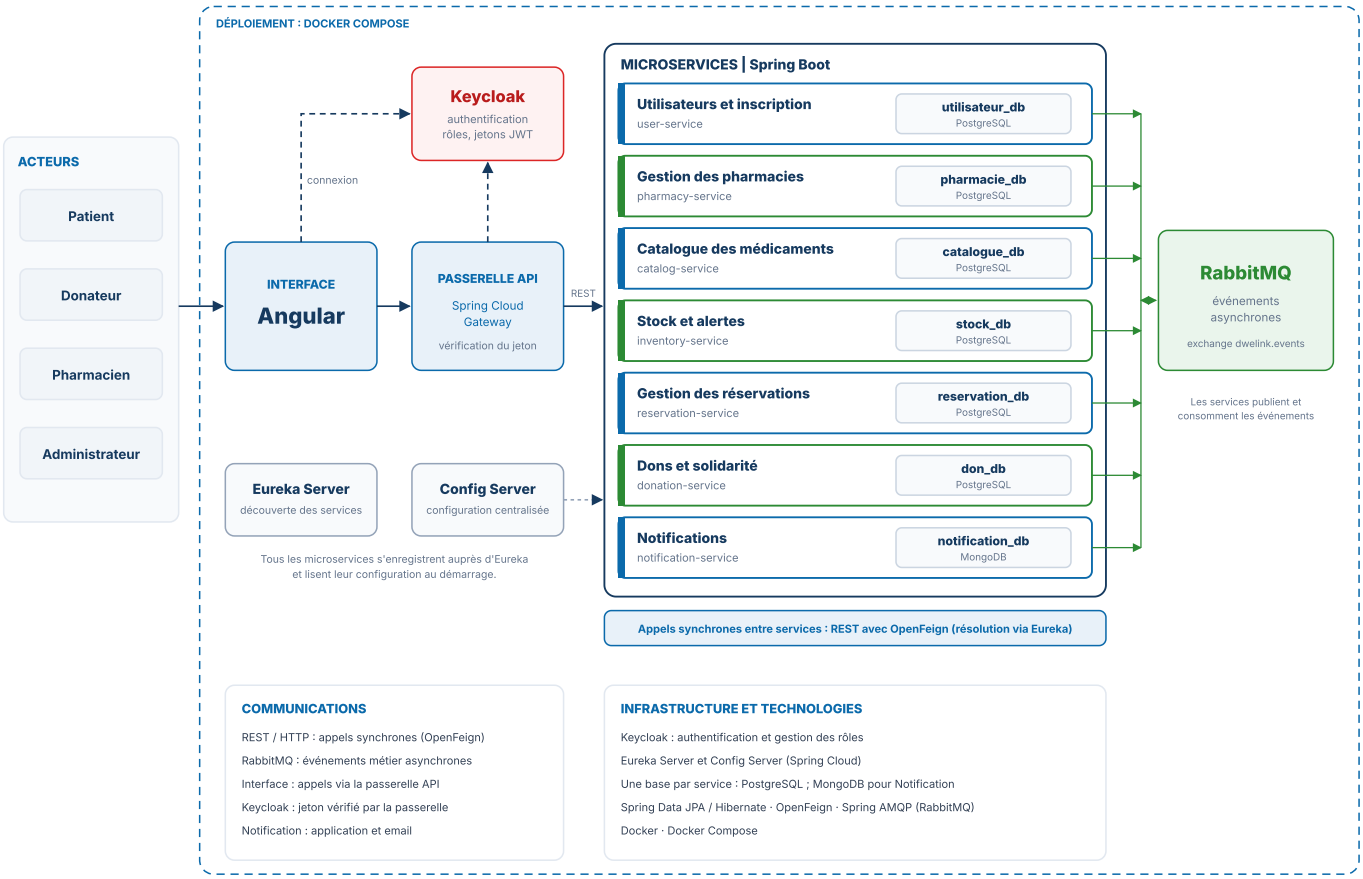
Les quatre acteurs héritent de l'acteur « Utilisateur » (connexion, profil, notifications). Trois relations « include » montrent les étapes obligatoires : une réservation vérifie toujours la disponibilité, un don vérifie toujours son éligibilité, et un don contrôlé ajoute toujours un lot au stock.



SECTION 03

03 Diagramme d'architecture globale

Interface Angular, passerelle API, Keycloak, Eureka, Config Server, sept microservices, leurs bases de données et RabbitMQ, déployés avec Docker Compose.



SECTION 04

04 Sept microservices complémentaires

Six services attribués individuellement et un service de notifications commun à l'équipe.

PRINCIPE DU DÉCOUPAGE

Chaque microservice correspond à un besoin métier précis. Il possède ses propres données et peut être développé, testé et lancé séparément. Les services ne partagent pas de base de données : ils communiquent par des appels REST lorsqu'une réponse immédiate est nécessaire, et par des messages RabbitMQ pour le reste.

CONTENU D'UNE FICHE

01 Responsabilités du service	02 Données gérées	03 API REST et rôles
04 Événements RabbitMQ	05 Appels REST	06 Gestion des pannes

01 Utilisateurs et inscription

user-service

RESPONSABLE
Hichem Jemail

BASE DE DONNÉES	ENDPOINTS REST	ÉVÉNEMENTS ÉMIS	ENTITÉS
utilisateur_db PostgreSQL · Spring Data JPA, Hibernate	**10**	**3**	**3**

RESPONSABILITÉS

- Créer et modifier le profil de chaque utilisateur
- Recevoir les demandes d'inscription des pharmaciens et leurs justificatifs
- Permettre à l'administrateur d'approuver ou de refuser ces demandes
- Activer, suspendre ou désactiver un compte
- La connexion et les rôles sont gérés par Keycloak

API REST

Méthode	URI	Rôle	Description	Codes HTTP
POST	`/api/utilisateurs/moi`	**Tous**	Créer son profil après l'inscription Keycloak	201, 400, 409
GET	`/api/utilisateurs/moi`	**Tous**	Consulter son profil	200, 404
PATCH	`/api/utilisateurs/moi`	**Tous**	Modifier son profil et ses préférences	200, 400
POST	`/api/demandes-pharmacien`	**Tous**	Déposer une demande d'inscription de pharmacien	201, 400, 409
POST	`/api/demandes-pharmacien/{id}/justificatifs`	**Tous**	Joindre un justificatif	201, 400, 404
GET	`/api/demandes-pharmacien?statut=`	**ADMIN**	Lister les demandes à examiner	200, 403
PATCH	`/api/demandes-pharmacien/{id}/decision`	**ADMIN**	Approuver ou refuser une demande	200, 403, 404, 409, 503
PATCH	`/api/utilisateurs/{id}/statut`	**ADMIN**	Suspendre ou réactiver un compte	200, 403, 404
GET	`/api/utilisateurs/{id}`	**Services internes**	Statut d'un utilisateur (appel de Pharmacies)	200, 404
GET	`/api/utilisateurs/{id}/contact`	**Services internes**	Email du destinataire (appel de Notifications)	200, 404

ÉVÉNEMENTS RABBITMQ (asynchrone)

Émet

- **DemandePharmacienDeposee** → Notifications : Nouvelle demande : prévenir l'administrateur
- **DemandePharmacienDecidee** → Notifications : Demande approuvée ou refusée : informer le pharmacien
- **StatutCompteModifie** → Pharmacies, Notifications : Compte suspendu : retirer ses rattachements

Consomme

Aucun

INTERACTIONS REST (synchrone)

Appelle

- **Keycloak** : attribuer le rôle PHARMACIEN, activer ou désactiver un compte

Est appelé par

- **Pharmacies** : vérifier qu'un pharmacien est validé
- **Notifications** : récupérer l'email du destinataire

GESTION DES PANNES

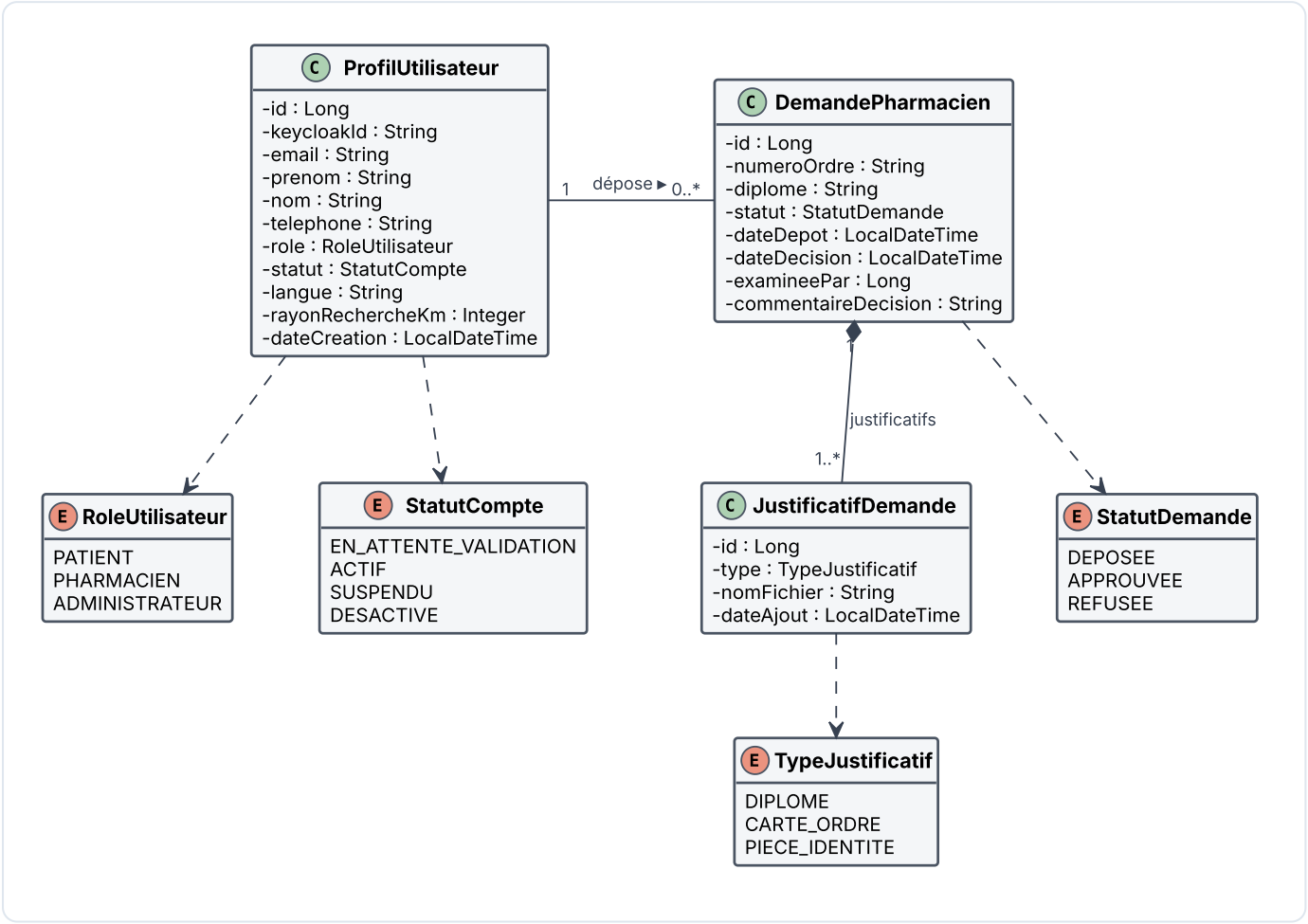
- Keycloak ne répond pas : erreur 503, la demande reste en attente
- RabbitMQ ne répond pas : l'action est enregistrée, la notification part plus tard

MODULE 01 · DIAGRAMME DE CLASSES

Utilisateurs et inscription

user-service

PostgreSQL • utilisateur_db



CLASSES PRINCIPALES (utilisateur_db)

- **ProfilUtilisateur** : identité, rôle, statut du compte, préférences
- **DemandePharmacien** : demande d'inscription, décision et examinateur
- **JustificatifDemande** : diplôme, carte de l'Ordre ou pièce d'identité

RELATIONS ET CARDINALITÉS

- **ProfilUtilisateur** 1 — 0..* **DemandePharmacien** : un utilisateur peut déposer plusieurs demandes
- **DemandePharmacien** 1 ◆ 1..* **JustificatifDemande** : composition, un justificatif n'existe pas sans sa demande

Énumérations : RoleUtilisateur, StatutCompte, TypeJustificatif, StatutDemande

02 Gestion des pharmacies

pharmacy-service

RESPONSABLE
Mohamed Yassine Ben Aïssa

BASE DE DONNÉES	ENDPOINTS REST	ÉVÉNEMENTS ÉMIS	ENTITÉS
pharmacie_db PostgreSQL · Spring Data JPA, Hibernate	12	3	4

RESPONSABILITÉS

- Déclarer une pharmacie (son créateur en devient le titulaire)
- Valider ou suspendre une pharmacie (administrateur)
- Rechercher les pharmacies proches ou de garde
- Gérer les horaires et les gardes
- Gérer les pharmaciens membres d'une pharmacie

RÈGLES MÉTIER

seule une pharmacie VALIDEE apparaît dans les recherches · seul le titulaire modifie la pharmacie et ses membres · le titulaire ne peut pas être retiré.

API REST

Méthode	URI	Rôle	Description	Codes HTTP
POST	`/api/pharmacies`	PHARMACIEN	Déclarer une pharmacie	201, 400, 409
GET	`/api/pharmacies/{id}`	Tous	Consulter une pharmacie	200, 404
PUT	`/api/pharmacies/{id}`	TITULAIRE	Modifier les informations	200, 400, 403, 404
GET	`/api/pharmacies/proches?lat=&lng=&rayonKm=`	Tous	Pharmacies validées proches	200, 400
GET	`/api/pharmacies?ville=&deGarde=`	Tous	Rechercher par ville ou pharmacies de garde	200
PATCH	`/api/pharmacies/{id}/statut`	ADMIN	Valider ou suspendre une pharmacie	200, 403, 404, 409
PUT	`/api/pharmacies/{id}/horaires`	TITULAIRE	Remplacer les horaires de la semaine	200, 400, 403
POST	`/api/pharmacies/{id}/gardes`	TITULAIRE	Ajouter une garde	201, 400, 403, 409
POST	`/api/pharmacies/{id}/membres`	PHARMACIEN	Demander à rejoindre la pharmacie	201, 403, 409
PATCH	`/api/pharmacies/{id}/membres/{membreId}`	TITULAIRE	Accepter ou refuser un membre	200, 403, 404, 409
DELETE	`/api/pharmacies/{id}/membres/{membreId}`	TITULAIRE	Retirer un membre	204, 403, 404
GET	`/api/pharmacies/{id}/membres/verifier?utilisateurId=`	Services internes	Savoir si un utilisateur est membre actif	200, 404

ÉVÉNEMENTS RABBITMQ (asynchrone)

Émet

- **StatutPharmacieModifie** → Stock, Réservations, Dons, Notifications : Pharmacie validée ou suspendue
- **DemandeRattachementCreee** → Notifications : Un pharmacien demande à rejoindre : prévenir le titulaire
- **MembresPharmacieModifies** → Notifications : Membre accepté, refusé ou retiré

Consomme

- **StatutCompteModifie** (Utilisateurs) : compte suspendu : ses lignes de membre sont retirées

INTERACTIONS REST (synchrone)

Appelle

- **Utilisateurs** : GET /api/utilisateurs/{id} : vérifier que le pharmacien est validé

Est appelé par

- **Stock, Réservations, Dons** : GET /{id} et /membres/verifier
- **Interface Angular** : via la passerelle API

GESTION DES PANNES

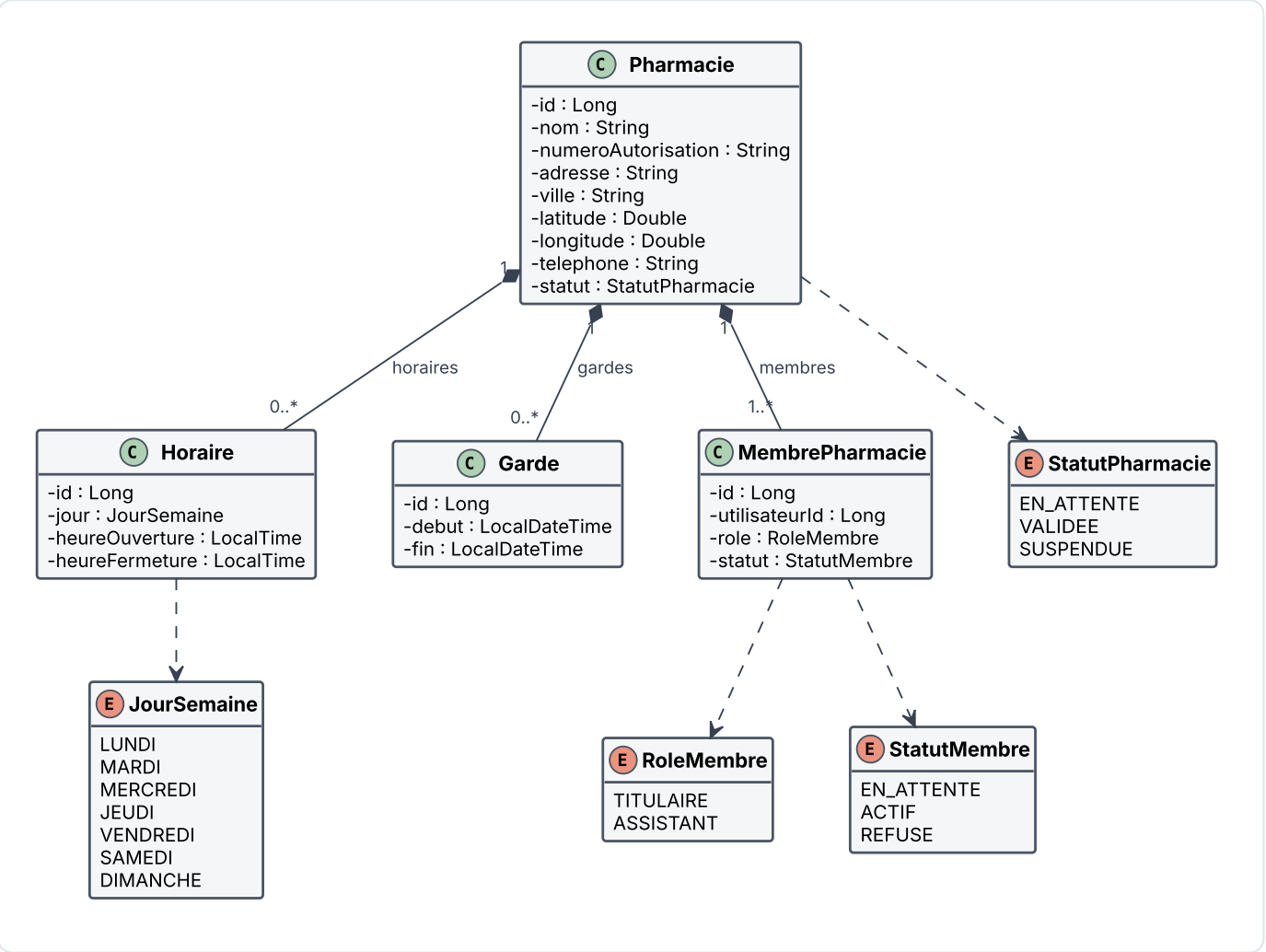
- user-service ne répond pas : erreur 503, rien n'est créé
- Message reçu deux fois : il est reconnu et ignoré
- Service arrêté : les messages attendent dans RabbitMQ et sont traités au redémarrage

MODULE 02 · DIAGRAMME DE CLASSES

Gestion des pharmacies

pharmacy-service

PostgreSQL • pharmacie_db



CLASSES PRINCIPALES (pharmacie_db)

- **Pharmacie** : informations, coordonnées (latitude, longitude), statut
- **Horaire** : jour, heure d'ouverture et de fermeture
- **Garde** : début et fin d'une garde
- **MembrePharmacie** : pharmacien rattaché, rôle et statut

RELATIONS ET CARDINALITÉS

- **Pharmacie** 1 ◆ 0..* **Horaire** : les horaires de la semaine
- **Pharmacie** 1 ◆ 0..* **Garde** : les périodes de garde
- **Pharmacie** 1 ◆ 1..* **MembrePharmacie** : au moins le titulaire

Énumérations : StatutPharmacie, JourSemaine, RoleMembre, StatutMembre

03 Catalogue des médicaments

catalog-service

RESPONSABLE
Wejdene Bouali

BASE DE DONNÉES	ENDPOINTS REST	ÉVÉNEMENTS ÉMIS	ENTITÉS
catalogue_db PostgreSQL · Spring Data JPA, Hibernate	**10**	**1**	**3**

RESPONSABILITÉS

- Gérer la fiche de chaque médicament : nom, code-barres, forme, catégorie, laboratoire
- Indiquer si une ordonnance est requise, la conservation et si c'est un stupéfiant
- Gérer les principes actifs et la composition de chaque médicament (dosage, unité)
- Rechercher un médicament par nom, catégorie ou laboratoire
- Proposer les médicaments équivalents
- Retirer un médicament du marché en enregistrant sa date de retrait

RÈGLES MÉTIER

un médicament contient au moins une composition · le code-barres est unique · équivalent = même forme et mêmes principes actifs avec les mêmes dosages · un médicament RETIRE garde sa fiche mais ne peut plus être stocké, réservé ni donné.

API REST

Méthode	URI	Rôle	Description	Codes HTTP
GET	`/api/medicaments?q=&categorie=&laboratoire=`	Tous	Rechercher des médicaments	200
GET	`/api/medicaments/{id}`	Tous	Consulter un médicament et sa composition	200, 404
GET	`/api/medicaments/{id}/equivalents`	Tous	Consulter les médicaments équivalents	200, 404
POST	`/api/medicaments`	ADMIN	Créer un médicament avec sa composition	201, 400, 403, 409
PUT	`/api/medicaments/{id}`	ADMIN	Modifier la fiche d'un médicament	200, 400, 403, 404
PATCH	`/api/medicaments/{id}/retrait`	ADMIN	Retirer du marché (statut RETIRE, date)	200, 403, 404, 409
POST	`/api/medicaments/{id}/compositions`	ADMIN	Ajouter un principe actif et son dosage	201, 400, 403, 404
DELETE	`/api/medicaments/{id}/compositions/{compositionId}`	ADMIN	Retirer une composition (pas la dernière)	204, 403, 404, 409
GET	`/api/principes-actifs?q=`	Tous	Rechercher un principe actif	200
POST	`/api/principes-actifs`	ADMIN	Créer un principe actif	201, 400, 403, 409

ÉVÉNEMENTS RABBITMQ (asynchrone)

Émet

- **MedicamentRetire** → Stock, Réservations, Dons : Retrait du marché : bloquer lots, réservations et dons

Consomme

Aucun

INTERACTIONS REST (synchrone)

Appelle

Aucun appel synchrone

Est appelé par

- **Stock, Réservations** : vérifier qu'un médicament existe et n'est pas retiré
- **Dons** : conservation et stupéfiant pour l'éligibilité
- **Interface Angular** : recherche et administration

GESTION DES PANNES

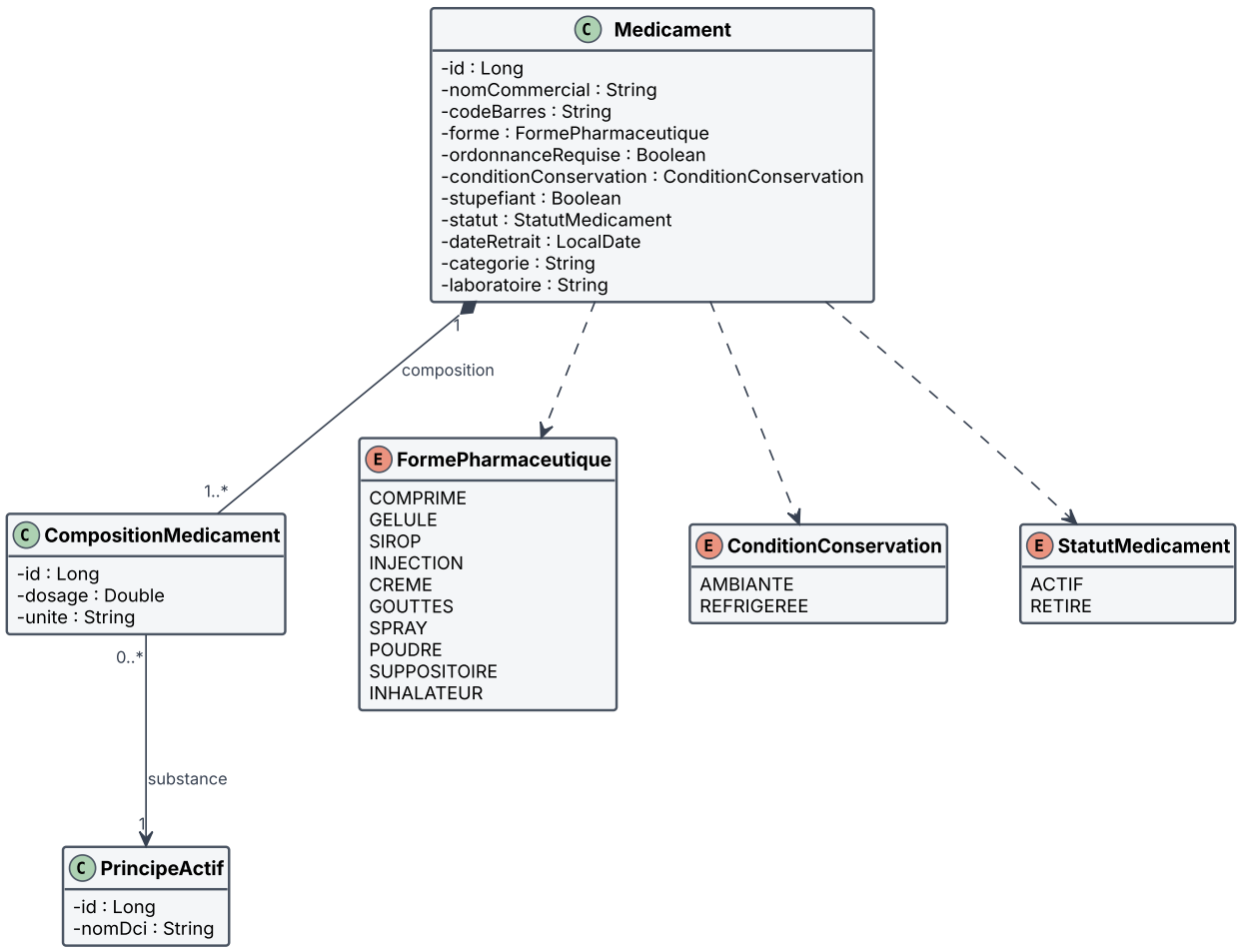
- Le catalogue n'appelle aucun autre service
- S'il ne répond pas, les services qui l'appellent renvoient une erreur 503

MODULE 03 · DIAGRAMME DE CLASSES

Catalogue des médicaments

catalog-service

PostgreSQL • catalogue_db



CLASSES PRINCIPALES (catalogue_db)

- **Medicament** : fiche du médicament : forme, catégorie, laboratoire, ordonnance, conservation, statut, date de retrait
- **CompositionMedicament** : dosage et unité d'un principe actif dans un médicament
- **PrincipeActif** : substance active, identifiée par son nom DCI

RELATIONS ET CARDINALITÉS

- **Medicament** 1 ◆ 1..* **CompositionMedicament** : composition, un médicament contient au moins un principe actif
- **CompositionMedicament** 0..* → 1 **PrincipeActif** : substance, un même principe actif peut entrer dans plusieurs compositions
- La catégorie et le laboratoire sont de simples attributs texte du médicament

Énumérations : FormePharmaceutique, ConditionConservation, StatutMedicament

04 Stock et alertes

inventory-service

RESPONSABLE
Rayen Sassi

BASE DE DONNÉES	ENDPOINTS REST	ÉVÉNEMENTS ÉMIS	ENTITÉS
stock_db PostgreSQL · Spring Data JPA, Hibernate	9	3	3

RESPONSABILITÉS

- Gérer le stock de chaque pharmacie : médicaments vendus, prix et seuil minimal
- Enregistrer les lots reçus avec leur date de péremption
- Garder l'historique des entrées et des sorties
- Mettre de côté les boîtes d'une réservation, puis les libérer ou les sortir
- Indiquer les pharmacies proches qui ont un médicament
- Prévenir la pharmacie en cas de stock faible, de rupture ou de péremption proche

RÈGLES MÉTIER

stock faible quand la quantité disponible est inférieure ou égale au seuil minimal · péremption proche à moins de 90 jours · un lot BLOQUE ou PERIME n'est ni vendu ni réservé · un lot d'origine DON n'est jamais vendu.

API REST

Méthode	URI	Rôle	Description	Codes HTTP
POST	`/api/articles-stock`	PHARMACIEN	Ajouter un médicament au stock	201, 400, 403, 409
GET	`/api/articles-stock?pharmacieId=`	PHARMACIEN	Voir le stock de sa pharmacie	200, 403
PATCH	`/api/articles-stock/{id}`	PHARMACIEN	Modifier le prix ou le seuil	200, 400, 403, 404
POST	`/api/articles-stock/{id}/lots`	PHARMACIEN	Enregistrer un lot reçu	201, 400, 403, 409
PATCH	`/api/lots/{id}/statut`	PHARMACIEN	Bloquer ou débloquer un lot	200, 403, 404
POST	`/api/articles-stock/{id}/mouvements`	PHARMACIEN	Enregistrer une vente ou un ajustement	201, 400, 403, 409
GET	`/api/articles-stock/{id}/mouvements`	PHARMACIEN	Historique des mouvements	200, 403
GET	`/api/disponibilite?medicamentId=&lat=&lng=`	Tous	Pharmacies proches qui ont le médicament	200, 400
POST	`/api/stock/reservations`	Services internes	Mettre de côté des boîtes (appel de Réservations)	201, 409

ÉVÉNEMENTS RABBITMQ (asynchrone)

Émet

- **StockFaible** → Notifications : Stock faible : alerter la pharmacie
- **RuptureStock** → Notifications : Rupture : alerter la pharmacie
- **PeremptionProche** → Notifications : Lot proche de la péremption

Consomme

- **ReservationRefusee / Annulee / Expiree** (Réservations) : libérer les boîtes
- **ReservationTerminee** (Réservations) : sortir les boîtes du stock
- **DonRecu / RemiseEffectuee** (Dons) : ajouter un lot d'origine DON, puis le sortir
- **MedicamentRetire** (Catalogue) : passer les lots au statut BLOQUE
- **StatutPharmacieModifie** (Pharmacies) : cacher le stock d'une pharmacie suspendue

INTERACTIONS REST (synchrone)

Appelle

- **Catalogue** : vérifier le médicament ajouté au stock
- **Pharmacies** : trouver les pharmacies proches

Est appelé par

- **Réservations** : mettre de côté les boîtes
- **Interface Angular** : via la passerelle API

GESTION DES PANNES

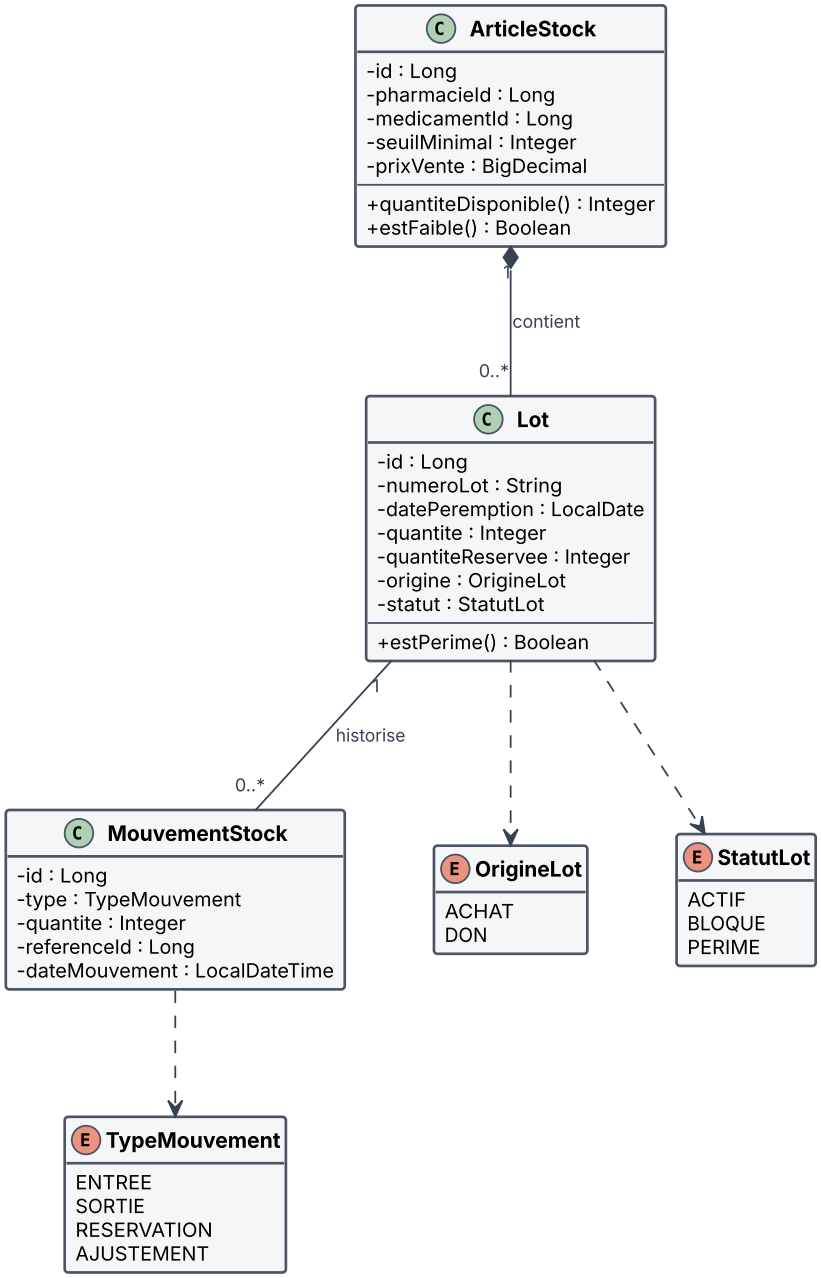
- Catalogue ou Pharmacies ne répond pas : erreur 503, le stock déjà enregistré reste utilisable
- Message reçu deux fois : il est reconnu et ignoré, aucun lot n'est créé en double

MODULE 04 · DIAGRAMME DE CLASSES

Stock et alertes

inventory-service

PostgreSQL • stock_db



CLASSES PRINCIPALES (stock_db)

- **ArticleStock** : médicament vendu par une pharmacie, avec son prix et son seuil
- **Lot** : numéro, date de péremption, quantités, origine (achat ou don) et statut
- **MouvementStock** : entrée, sortie, réservation ou ajustement

RELATIONS ET CARDINALITÉS

- **ArticleStock** 1 ◆ 0..* **Lot** : un article contient ses lots
- **Lot** 1 — 0..* **MouvementStock** : chaque lot garde l'historique de ses mouvements

Énumérations : OrigineLot, StatutLot, TypeMouvement

05 Gestion des réservations

reservation-service

RESPONSABLE
Roua Selmi

BASE DE DONNÉES	ENDPOINTS REST	ÉVÉNEMENTS ÉMIS	ENTITÉS
reservation_db PostgreSQL · Spring Data JPA, Hibernate	9	6	3

RESPONSABILITÉS

- Créer une réservation en mettant de côté les boîtes demandées
- Confirmer (avec un code de retrait) ou refuser une réservation
- Annuler une réservation tant qu'elle n'est pas retirée
- Expirer automatiquement les réservations hors délai
- Tracer chaque changement de statut (acteur, motif, date)

RÈGLES MÉTIER

3 réservations actives au maximum · réponse de la pharmacie sous 2 h · retrait sous 24 h · ordonnance vérifiée au comptoir si requise.

API REST

Méthode	URI	Rôle	Description	Codes HTTP
POST	`/api/reservations`	PATIENT	Créer une réservation	201, 400, 404, 409, 503
GET	`/api/reservations/{id}`	PATIENT / PHARM.	Consulter une réservation	200, 403, 404
GET	`/api/reservations/mes-reservations?statut=`	PATIENT	Mes réservations	200
GET	`/api/reservations?pharmacieId=&statut=`	PHARMACIEN	Réservations de la pharmacie	200, 403
PATCH	`/api/reservations/{id}/confirmer`	PHARMACIEN	Confirmer et générer le code	200, 403, 404, 409
PATCH	`/api/reservations/{id}/refuser`	PHARMACIEN	Refuser avec motif	200, 400, 403, 404, 409
PATCH	`/api/reservations/{id}/annuler`	PATIENT	Annuler	200, 403, 404, 409
PATCH	`/api/reservations/{id}/terminer`	PHARMACIEN	Valider le retrait avec le code	200, 400, 403, 404, 409
GET	`/api/reservations/{id}/historique`	PATIENT / PHARM.	Historique des statuts	200, 403, 404

ÉVÉNEMENTS RABBITMQ (asynchrone)

Émet

- **ReservationCreee** → Notifications : Nouvelle réservation : prévenir la pharmacie
- **ReservationConfirmee** → Notifications : Envoyer le code de retrait au patient
- **ReservationRefusee** → Stock, Notifications : Libérer les boîtes, informer le patient
- **ReservationAnnulee** → Stock, Notifications : Libérer les boîtes, informer la pharmacie
- **ReservationExpiree** → Stock, Notifications : Libérer les boîtes, informer
- **ReservationTerminee** → Stock : Retrait effectué : sortie du stock

Consomme

- **StatutPharmacieModifie** (Pharmacies) : annuler les réservations actives
- **MedicamentRetire** (Catalogue) : refuser les réservations en attente

INTERACTIONS REST (synchrone)

Appelle

- **Catalogue** : statut du médicament et ordonnance requise
- **Pharmacies** : pharmacie validée et vérification des membres
- **Stock** : mettre de côté les boîtes demandées

Est appelé par

- **Interface Angular** : espaces patient et pharmacien

GESTION DES PANNES

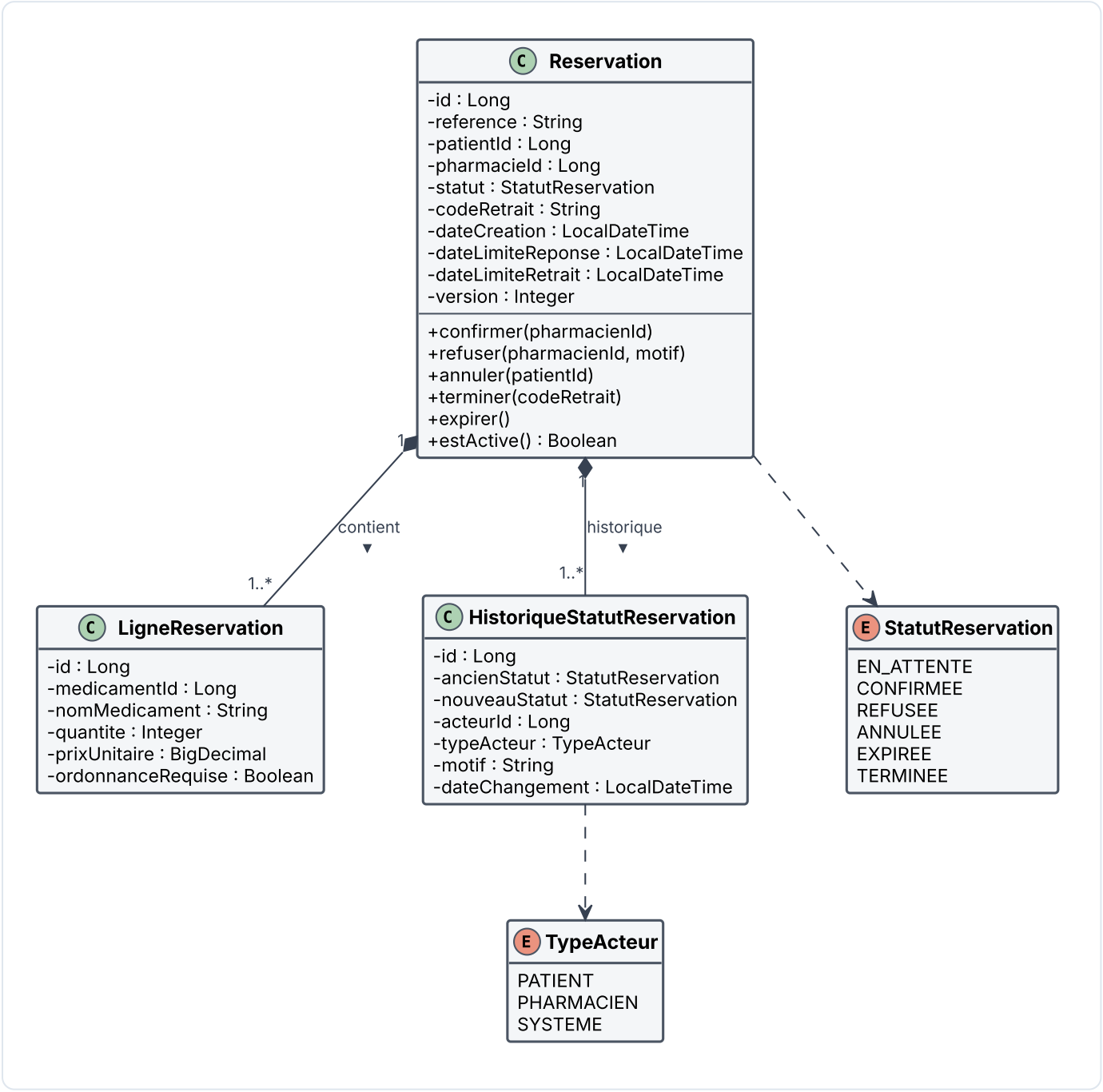
- Stock ne répond pas : erreur 503, la réservation n'est pas créée
- RabbitMQ ne répond pas : la réservation reste valide, la notification part plus tard

MODULE 05 · DIAGRAMME DE CLASSES

Gestion des réservations

reservation-service

PostgreSQL • reservation_db



CLASSES PRINCIPALES (reservation_db)

- **Reservation** : patient, pharmacie, statut, code, échéances
- **LigneReservation** : médicament, quantité, prix copié
- **HistoriqueStatutReservation** : changement de statut, acteur, motif

RELATIONS ET CARDINALITÉS

- **Reservation** 1 ◆ 1..* **LigneReservation** : une ligne par médicament
- **Reservation** 1 ◆ 1..* **HistoriqueStatutReservation** : une entrée par changement de statut

Énumérations : StatutReservation, TypeActeur

06 Dons et solidarité

donation-service

RESPONSABLE
Mohamed Yassine Azzouz

BASE DE DONNÉES	ENDPOINTS REST	ÉVÉNEMENTS ÉMIS	ENTITÉS
don_db PostgreSQL · Spring Data JPA, Hibernate	8	5	3

RESPONSABILITÉS

- Permettre au donateur de proposer un don et d'en suivre le statut
- Vérifier que le médicament peut être donné et que la pharmacie est validée
- Permettre à la pharmacie d'accepter, de contrôler puis de remettre le don
- Garantir qu'un médicament donné n'est jamais vendu
- Informer le stock à la réception et à chaque remise

RÈGLES D'ÉLIGIBILITÉ D'UN DON

au moins 6 mois avant la péremption · boîte jamais ouverte · pas de conservation au froid · pas de stupéfiant · médicament non retiré du marché.

API REST

Méthode	URI	Rôle	Description	Codes HTTP
POST	`/api/dons`	Tous	Proposer un don (donateur)	201, 400, 403, 422, 503
GET	`/api/dons/{id}`	Tous	Consulter un don (donateur ou pharmacie concernée)	200, 403, 404
GET	`/api/dons/mes-dons`	Tous	Suivre mes dons (donateur)	200, 403
PATCH	`/api/dons/{id}/annuler`	Tous	Annuler un don (donateur)	200, 403, 404, 409
GET	`/api/dons?pharmacieId=&statut=&page=`	PHARMACIEN	Dons reçus par la pharmacie	200, 403
PATCH	`/api/dons/{id}/decision`	PHARMACIEN	Accepter ou refuser un don	200, 400, 403, 404, 409
POST	`/api/dons/{id}/controle`	PHARMACIEN	Contrôler le don au dépôt	201, 400, 403, 404, 409
POST	`/api/dons/{id}/remises`	PHARMACIEN	Remettre gratuitement le médicament	201, 400, 403, 404, 409

ÉVÉNEMENTS RABBITMQ (asynchrone)

Émet

- **DonPropose** → Notifications : Nouveau don : prévenir la pharmacie
- **DonDecide** → Notifications : Don accepté ou refusé (y compris au contrôle) : prévenir le donateur
- **DonAnnule** → Notifications : Don annulé : prévenir le donateur et la pharmacie
- **DonRecu** → Stock : Don contrôlé : ajouter le lot au stock solidaire
- **RemiseEffectuee** → Stock : Remise gratuite : décrémenter le stock solidaire

Consomme

- **MedicamentRetire** (Catalogue) : annuler les dons en attente ou acceptés
- **StatutPharmacieModifie** (Pharmacies) : pharmacie suspendue : annuler ses dons en attente

INTERACTIONS REST (synchrone)

Appelle

- **Catalogue** : vérifier que le médicament peut être donné
- **Pharmacies** : vérifier que la pharmacie est validée

Est appelé par

- **Interface Angular** : espaces donateur et pharmacien

GESTION DES PANNES

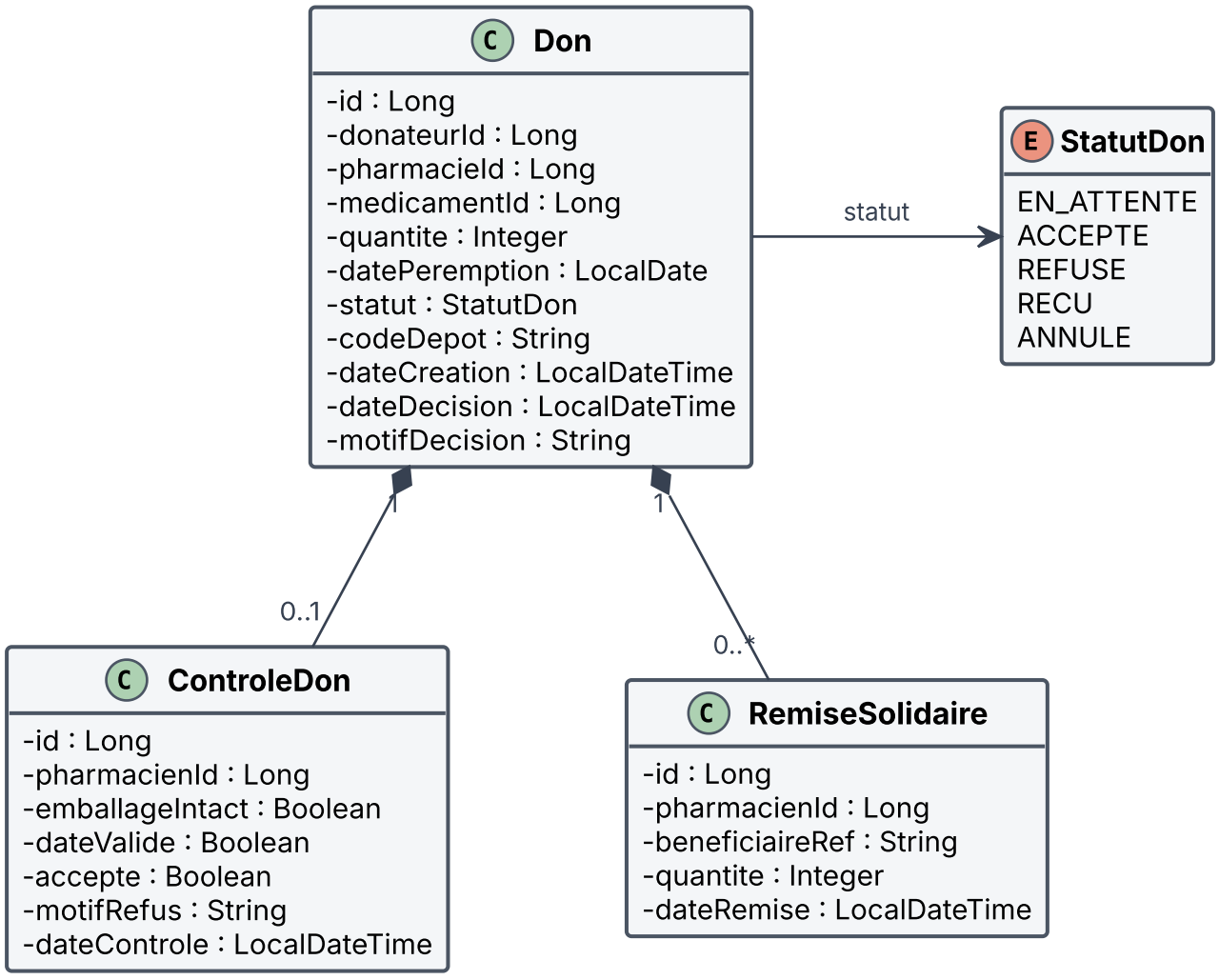
- Catalogue ou Pharmacies ne répond pas : erreur 503, le don n'est pas enregistré
- Stock arrêté au dépôt : le message DonRecu attend dans RabbitMQ

MODULE 06 · DIAGRAMME DE CLASSES

Dons et solidarité

donation-service

PostgreSQL • don_db



CLASSES PRINCIPALES (don_db)

- **Don** : médicament proposé, quantité, péremption, statut, code de dépôt, décision
- **ControleDon** : vérification faite au dépôt (0..1 par don)
- **RemiseSolidaire** : remise gratuite à un bénéficiaire ; la somme des remises ne dépasse jamais la quantité du don

RELATIONS ET CARDINALITÉS

- **Don** 1 ◆ 0..1 **ControleDon** : un seul contrôle au dépôt
- **Don** 1 ◆ 0..* **RemiseSolidaire** : un don peut être remis en plusieurs fois

Énumérations : StatutDon

07 Notifications

notification-service

RESPONSABLE
Microservice commun

BASE DE DONNÉES	ENDPOINTS REST	ÉVÉNEMENTS ÉMIS	ENTITÉS
notification_db MongoDB · Spring Data MongoDB	5	0	2

RESPONSABILITÉS

- Recevoir les événements des autres services via RabbitMQ
- Créer la notification correspondante
- L'envoyer dans l'application et par email
- Gérer son statut (lue ou non lue) et les préférences de l'utilisateur

API REST

Méthode	URI	Rôle	Description	Codes HTTP
GET	`/api/notifications?lue=`	Tous	Lister mes notifications	200, 401
GET	`/api/notifications/non-lues/nombre`	Tous	Nombre de notifications non lues	200, 401
PUT	`/api/notifications/{id}/lue`	Tous	Marquer comme lue	200, 403, 404
PUT	`/api/notifications/tout-lu`	Tous	Tout marquer comme lu	204, 401
PUT	`/api/notifications/preferences`	Tous	Modifier mes préférences	200, 400

ÉVÉNEMENTS RABBITMQ (asynchrone)

Émet

Aucun : le service est uniquement consommateur

Consomme

- **Événements des comptes** (Utilisateurs) : demande déposée, décision
- **Événements des pharmacies** (Pharmacies) : statut, rattachement, membres
- **Alertes de stock** (Stock) : stock faible, rupture, péremption proche
- **Événements des réservations** (Réservations) : création, confirmation, refus, annulation, expiration
- **Événements des dons** (Dons) : proposition, décision, annulation

INTERACTIONS REST (synchrone)

Appelle

- **Utilisateurs** : récupérer l'email du destinataire

Est appelé par

- **Interface Angular** : liste des notifications

GESTION DES PANNES

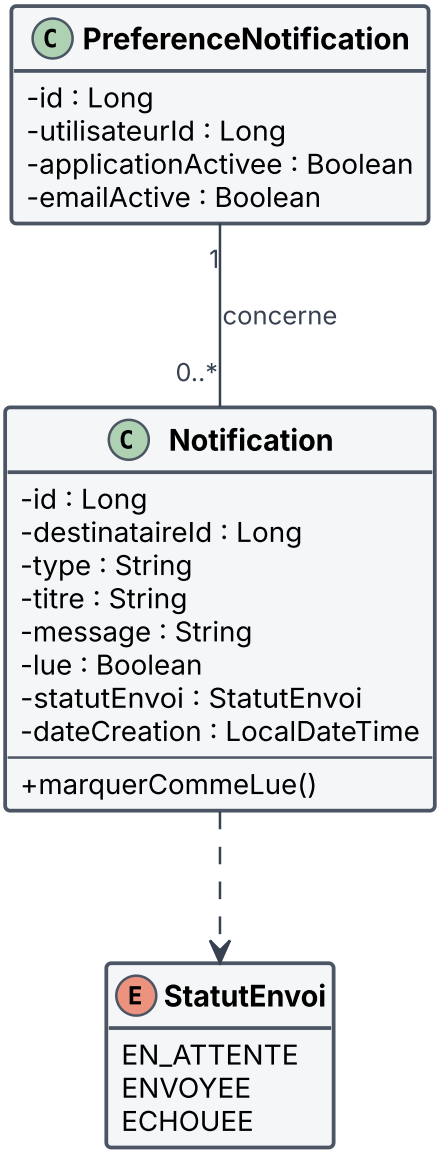
- RabbitMQ ne répond pas : les messages attendent et sont traités ensuite
- Email non envoyé : 3 nouveaux essais ; la notification reste visible
- Service arrêté : les autres services continuent de fonctionner

MODULE 07 · DIAGRAMME DE CLASSES

Notifications

notification-service

MongoDB • notification_db



CLASSES PRINCIPALES (notification_db)

- **Notification** : destinataire, type, contenu, lue ou non
- **PreferenceNotification** : canaux choisis par l'utilisateur

RELATIONS ET CARDINALITÉS

- **PreferenceNotification** 1 — 0..* **Notification** : les préférences d'un utilisateur s'appliquent à ses notifications

Énumérations : StatutEnvoi

SECTION 05

05 Des données isolées par service

Chaque service possède sa propre base de données et ne lit jamais celle d'un autre.



utilisateur_db
Utilisateurs et inscription

pharmacie_db
Gestion des pharmacies

catalogue_db
Catalogue des médicaments

stock_db
Stock et alertes

reservation_db
Gestion des réservations

don_db
Dons et solidarité

notification_db
Notifications

Base	Service	Technologie	Pourquoi ce choix
utilisateur_db	Utilisateurs et inscription	PostgreSQL	Données liées entre elles : profils, demandes et justificatifs.
pharmacie_db	Gestion des pharmacies	PostgreSQL	Pharmacies, horaires, gardes et membres reliés entre eux.
catalogue_db	Catalogue des médicaments	PostgreSQL	Un médicament contient plusieurs principes actifs : relations à gérer.
stock_db	Stock et alertes	PostgreSQL	Les transactions évitent de réserver deux fois la même boîte.
reservation_db	Gestion des réservations	PostgreSQL	Une réservation et ses lignes sont enregistrées ensemble.
don_db	Dons et solidarité	PostgreSQL	Le suivi d'un don (contrôle, remises) doit rester cohérent.
notification_db	Notifications	MongoDB	Le contenu d'une notification change selon son type : format souple.

Quand un service a besoin d'une information d'un autre service, il garde seulement son identifiant (par exemple pharmacieId ou medicamentId) et demande le reste au service concerné.

SECTION 06

06 Interactions entre les services

Des appels REST pour les vérifications immédiates et des événements RabbitMQ pour les traitements asynchrones.

CARTE DES INTERACTIONS (LIGNE : ÉMETTEUR OU APPELANT ; COLONNE : DESTINATAIRE OU APPELÉ)

	Utilisateurs	Pharmacies	Catalogue	Stock	Réservations	Dons	Notifications
Utilisateurs		ÉVÉNEMENT					ÉVÉNEMENT
Pharmacies	REST			ÉVÉNEMENT	ÉVÉNEMENT	ÉVÉNEMENT	ÉVÉNEMENT
Catalogue				ÉVÉNEMENT	ÉVÉNEMENT	ÉVÉNEMENT	
Stock		REST	REST				ÉVÉNEMENT
Réservations		REST	REST	REST ÉVÉNEMENT			ÉVÉNEMENT
Dons		REST	REST	ÉVÉNEMENT			ÉVÉNEMENT
Notifications	REST						

REST : appel direct qui attend une réponse (OpenFeign). ÉVÉNEMENT : message envoyé par RabbitMQ et traité plus tard.

COMMUNICATIONS SYNCHRONES (REST)

Appelant	Service appelé	Objet de l'appel	Endpoint cité
Pharmacies	Utilisateurs	Vérifier qu'un pharmacien est validé	`GET /api/utilisateurs/{id}`
Stock	Catalogue	Vérifier le médicament lors de l'ajout au stock	`GET /api/medicaments/{id}`
Stock	Pharmacies	Trouver les pharmacies proches	`GET /api/pharmacies/proches`
Réservations	Catalogue	Vérifier les médicaments réservés	`GET /api/medicaments/{id}`
Réservations	Pharmacies	Pharmacie validée et vérification des membres	`GET /api/pharmacies/{id}`
Réservations	Stock	Mettre de côté les boîtes demandées	`POST /api/stock/reservations`
Dons	Catalogue	Éligibilité : conservation, stupéfiant, statut	`GET /api/medicaments/{id}`
Dons	Pharmacies	Vérifier que la pharmacie est validée	`GET /api/pharmacies/{id}`
Notifications	Utilisateurs	Récupérer l'email du destinataire	`GET /api/utilisateurs/{id}/contact`
Interface Angular	Tous les services	Appels des trois espaces, via la passerelle API	`/api/**`

SECTION 06

06 Événements RabbitMQ

L'émetteur n'attend pas le traitement du consommateur.

Événement	Émetteur	Consommateurs	Usage
DemandePharmacienDeposee	Utilisateurs	Notifications	Nouvelle demande : prévenir l'administrateur
DemandePharmacienDecidee	Utilisateurs	Notifications	Demande approuvée ou refusée : informer le pharmacien
StatutCompteModifie	Utilisateurs	Pharmacies, Notifications	Compte suspendu : retirer ses rattachements
StatutPharmacieModifie	Pharmacies	Stock, Réservations, Dons, Notifications	Pharmacie validée ou suspendue
DemandeRattachementCreee	Pharmacies	Notifications	Un pharmacien demande à rejoindre : prévenir le titulaire
MembresPharmacieModifies	Pharmacies	Notifications	Membre accepté, refusé ou retiré
MedicamentRetire	Catalogue	Stock, Réservations, Dons	Retrait du marché : bloquer lots, réservations et dons
StockFaible	Stock	Notifications	Stock faible : alerter la pharmacie
RuptureStock	Stock	Notifications	Rupture : alerter la pharmacie
PeremptionProche	Stock	Notifications	Lot proche de la péremption
ReservationCreee	Réservations	Notifications	Nouvelle réservation : prévenir la pharmacie
ReservationConfirmee	Réservations	Notifications	Envoyer le code de retrait au patient
ReservationRefusee	Réservations	Stock, Notifications	Libérer les boîtes, informer le patient
ReservationAnnulee	Réservations	Stock, Notifications	Libérer les boîtes, informer la pharmacie
ReservationExpiree	Réservations	Stock, Notifications	Libérer les boîtes, informer
ReservationTerminee	Réservations	Stock	Retrait effectué : sortie du stock
DonPropose	Dons	Notifications	Nouveau don : prévenir la pharmacie
DonDecide	Dons	Notifications	Don accepté ou refusé (y compris au contrôle) : prévenir le donateur
DonAnnule	Dons	Notifications	Don annulé : prévenir le donateur et la pharmacie
DonRecu	Dons	Stock	Don contrôlé : ajouter le lot au stock solidaire
RemiseEffectuee	Dons	Stock	Remise gratuite : décrémenter le stock solidaire

SECTION 07

07 Organisation, planning et risques

Les autres informations nécessaires à la compréhension de la conception.

TECHNOLOGIES UTILISÉES

Domaine	Technologies
Interface	Angular
Serveur	Spring Boot, Spring Cloud (Gateway, Eureka, Config Server), API REST, Spring OpenFeign, Maven
Données	PostgreSQL avec Spring Data JPA et Hibernate (6 services) ; MongoDB (Notifications)
Messagerie	RabbitMQ (Spring AMQP)
Sécurité	Keycloak : authentification et gestion des rôles
Tests	JUnit, Mockito, Postman
Déploiement	Docker, Docker Compose, Git et GitHub

PLANNING PRÉVISIONNEL

Semaine	Phase	Tâches principales
S1	Analyse et conception	Cahier des charges, diagrammes UML, architecture microservices
S2	Configuration et services	Configuration Spring Cloud, Keycloak, RabbitMQ, création des microservices et des bases de données
S3	Développement	Fonctionnalités métier et interfaces Angular
S4	Intégration	Communication REST / OpenFeign et RabbitMQ, liaison Angular–microservices, Docker
S5	Tests et finalisation	Tests, correction des anomalies, documentation et préparation de la soutenance

DIAGRAMME DE GANTT SIMPLIFIÉ

Phase	S1	S2	S3	S4	S5
Analyse et conception	■				
Configuration et services		■			
Développement			■		
Intégration				■	
Tests et finalisation					■

DÉPÔT GIT

```
dwelink/
├── Backend/
│   ├── gateway-service/
│   ├── discovery-service/      (Eureka)
│   ├── config-service/
│   ├── user-service/
│   ├── pharmacy-service/
│   ├── catalog-service/
│   ├── inventory-service/
│   ├── reservation-service/
│   ├── donation-service/
│   └── notification-service/
├── Frontend/
│   └── dwelink-web/            (Angular)
├── Documentation/
│   ├── Presentation/
│   └── Diagrams/
├── docker-compose.yml
└── README.md
```

README.MD

- Sujet et objectifs du projet
- Membres de l'équipe et microservice de chacun
- Schéma d'architecture
- Prérequis et lancement : `docker compose up`

RÈGLES DE TRAVAIL

- Une branche par fonctionnalité, fusion par demande de fusion (pull request)
- Commits personnels réguliers de chaque membre
- Suivi des tâches dans GitHub Projects
- Chaque microservice compile et démarre dès la Validation 1

RISQUES IDENTIFIÉS

Risque	Impact	Solution prévue
Retard de développement	Élevé	Prioriser les fonctionnalités essentielles
Problèmes d'intégration entre microservices	Élevé	Tester les API progressivement avec Postman
Difficultés avec Spring Cloud, Keycloak et RabbitMQ	Moyen	Configurer l'infrastructure dès S2
Déséquilibre de la charge de travail	Moyen	Répartir les tâches selon leur complexité
Don de médicament non conforme	Moyen	Règles d'éligibilité et contrôle obligatoire par le pharmacien
Erreurs lors du déploiement	Moyen	Tester Docker et les communications avant S5